

# Test Cases Report

---

The following outlines the critical test scenarios needed to verify system correctness.

## 1. Cart Management Tests

### TC 1.01: Add Item to Cart

- **Preconditions:** Valid product exists with stock > 0.
- **Steps:** Call `Cart.addItem()` with a valid quantity (e.g., 1).
- **Expected Result:** Item is added to `Cart`. Item count increases. Total price reflects `Product.price * 1`.

### TC 1.02: Add Existing Item to Cart

- **Preconditions:** Cart already contains Product A (qty 1).
- **Steps:** Call `Cart.addItem()` with Product A (qty 2).
- **Expected Result:** Cart does not add a new item row. The existing `CartItem` quantity updates to 3.

### TC 1.03: Add Item Exceeding Stock

- **Preconditions:** Product A has 5 units in stock.
- **Steps:** Call `Cart.addItem()` for Product A with quantity 6.
- **Expected Result:** Exception thrown indicating insufficient stock. Cart state remains unchanged.

### TC 1.04: Add Item with Negative Quantity

- **Steps:** Call `Cart.addItem()` with quantity -1.
- **Expected Result:** `IllegalArgumentException` thrown.

### TC 1.05: Update Item Quantity

- **Preconditions:** Cart contains Product A (qty 2). Stock is 10.
- **Steps:** Call `Cart.updateQuantity()` for Product A with new quantity 5.
- **Expected Result:** Product A quantity in cart becomes 5.

## TC 1.06: Remove Item from Cart

- **Preconditions:** Cart contains Product A and B.
- **Steps:** Call `Cart.removeItem()` for Product A.
- **Expected Result:** Product A is no longer in the cart. Product B remains. Total is updated.

## 2. Pricing and Tax Tests

### TC 2.01: Calculate Subtotal

- **Preconditions:** Cart contains Product A (₹100, qty 2) and Product B (₹50, qty 1).
- **Steps:** Call `Cart.calculateSubtotal()`.
- **Expected Result:** Returns ₹250.00.

### TC 2.02: Calculate Tax

- **Preconditions:** Subtotal is ₹100. Tax rate is 10%. No coupons applied.
- **Steps:** Call `Cart.calculateTax()`.
- **Expected Result:** Returns ₹10.00.

## 3. Coupon Application Tests

### TC 3.01: Apply Valid Flat Coupon

- **Preconditions:** Subtotal is ₹500. Coupon FLAT100 is valid, ₹100 off.
- **Steps:** Apply coupon. Check totals.
- **Expected Result:** Discount is ₹100. Tax applies on ₹400. Final total calculates correctly.

### TC 3.02: Apply Valid Percentage Coupon

- **Preconditions:** Subtotal is ₹500. Coupon PERC10 is valid, 10% off.
- **Steps:** Apply coupon. Check totals.
- **Expected Result:** Discount is ₹50.

### TC 3.03: Apply Expired Coupon

- **Preconditions:** Coupon EXP is expired.
- **Steps:** Apply coupon to cart.

- **Expected Result:** `IllegalArgumentException` thrown ("Coupon expired").

## TC 3.04: Flat Coupon Exceeding Subtotal

- **Preconditions:** Subtotal is ₹50. Coupon is ₹100 FLAT.
- **Steps:** Apply coupon.
- **Expected Result:** Discount amount is exactly ₹50. Total (before tax) is ₹0. Negative totals are prevented.

## 4. Checkout Tests

### TC 4.01: Successful Checkout

- **Preconditions:** Cart has items. Stock is valid.
- **Steps:** Call `Cart.checkout()`.
- **Expected Result:** Returns receipt string. Cart is emptied. Product stock in memory is correctly reduced.

### TC 4.02: Checkout with Empty Cart

- **Steps:** Call `Cart.checkout()` on an empty cart.
- **Expected Result:** `IllegalStateException` thrown.

### TC 4.03: Stock Changes Before Checkout

- **Preconditions:** Item added to cart (qty 5). Before checkout, memory stock is manually reduced to 4 by another system.
- **Steps:** Call `Cart.checkout()`.
- **Expected Result:** Throws exception "Insufficient stock".